ReUse Campus

A Student Marketplace with an AI Chatbot Assistant

MILESTONE 1

Problem Statement and Design Document

Prepared by

Team member	Institute ID	GitHub username
Tarun Kumar	CS26MT012	cs26mt012
Chetan	CS26MT019	Chetan-cs26mt019
Archit Kumar	CS26MT020	ArchitKumar-020

Repository: github.com/Chetan-cs26mt019/reuseCampus

Revised concept: replace AI semantic search and similar-item recommendations with a conversational chatbot that helps students find items and understand marketplace features.

Document Guide

Section	What it covers
1. Project overview	Problem, objectives, scope, user roles and verification.
2. Features and sitemap	Marketplace functions and chatbot-centered navigation.
3. Dataset and reproducibility	Synthetic users, listings, conversations and test data.
4. User stories	Testable outcomes for the chatbot and marketplace.
5. Data model and lifecycle	Tables, relationships, status rules and privacy.
6. AI chatbot design	Conversation flow, intent handling, retrieval, guardrails and fallback.
7. Page-by-page design	Expected behaviour of each screen.
8. API, security and privacy	Technology choices, chatbot endpoints and access control.
9. Testing and deployment	Chatbot quality, safety, integration and acceptance criteria.
10. Team workflow and plan	Implementation plan and responsibilities.

What changed in this version

- The previous AI semantic-search and similar-item recommendation design has been removed as the proposed AI feature.
- The new AI feature is a chat-style assistant inside ReUse Campus. Students can ask natural-language questions and receive conversational guidance based on available listings and marketplace rules.
- The chatbot will use structured database filters for factual constraints such as price, category, condition and availability; it must not invent items or claim a listing is available unless the database confirms it.
- If the chatbot service is unavailable, students can still use the ordinary listing browser, filters and direct marketplace pages.
- Chatbot replies are treated as assistance, not as a replacement for seller confirmation, moderator decisions or verified marketplace records.

Implementation status note: this document describes the proposed design and acceptance criteria. It does not claim that the chatbot, dataset targets or tests have already been implemented or passed.

1. Project Overview, Scope and Roles

Project name: ReUse Campus — A Student Marketplace with an AI Chatbot Assistant

Problem statement

Students often need textbooks, electronics, hostel essentials and other items for a limited period. Buying everything new can be expensive, while informal class groups make it hard to compare listings or know whether an item is still available. ReUse Campus provides a campus-focused place where students can list, discover, enquire about and reserve second-hand items. Its AI chatbot gives students a simpler conversational way to ask for help, explore relevant listings and understand how to use the marketplace.

Objectives

- Allow students to register, sign in, manage their profile and request account deletion.
- Let students create, edit and browse listings with a title, description, price, condition, status and at least one photo.
- Support category, price, condition and availability filters through ordinary marketplace controls.
- Provide a conversational chatbot that can interpret a student's request and help find relevant listings using current marketplace data.
- Allow buyers and sellers to communicate through private enquiry threads and manage reservations.
- Provide favourites, listing reports and a moderator review process.
- Protect student information and ensure chatbot access follows the same permissions as the rest of the application.
- Deliver a tested, responsive application that can be started using Docker Compose.

Scope and exclusions

The first version will not process payments or provide delivery tracking. Buyers and sellers will arrange item handover themselves. The chatbot will not negotiate or complete a transaction on behalf of a user, guarantee product quality, verify a seller's claims, make moderation decisions, or provide unrestricted general-purpose advice. It will answer within the marketplace's intended scope and guide users to the relevant screen when an action requires authentication.

Roles

Role	Permissions
Visitor	Browse public available listings and ask general marketplace-navigation questions. Sign-in is required for protected actions.
Registered student	Manage their own profile and listings, save favourites, use private enquiries/messages, request reservations and view their own activity.
Moderator	Review reports, hide or remove listings when justified, and access moderation records. This is a single moderator role.

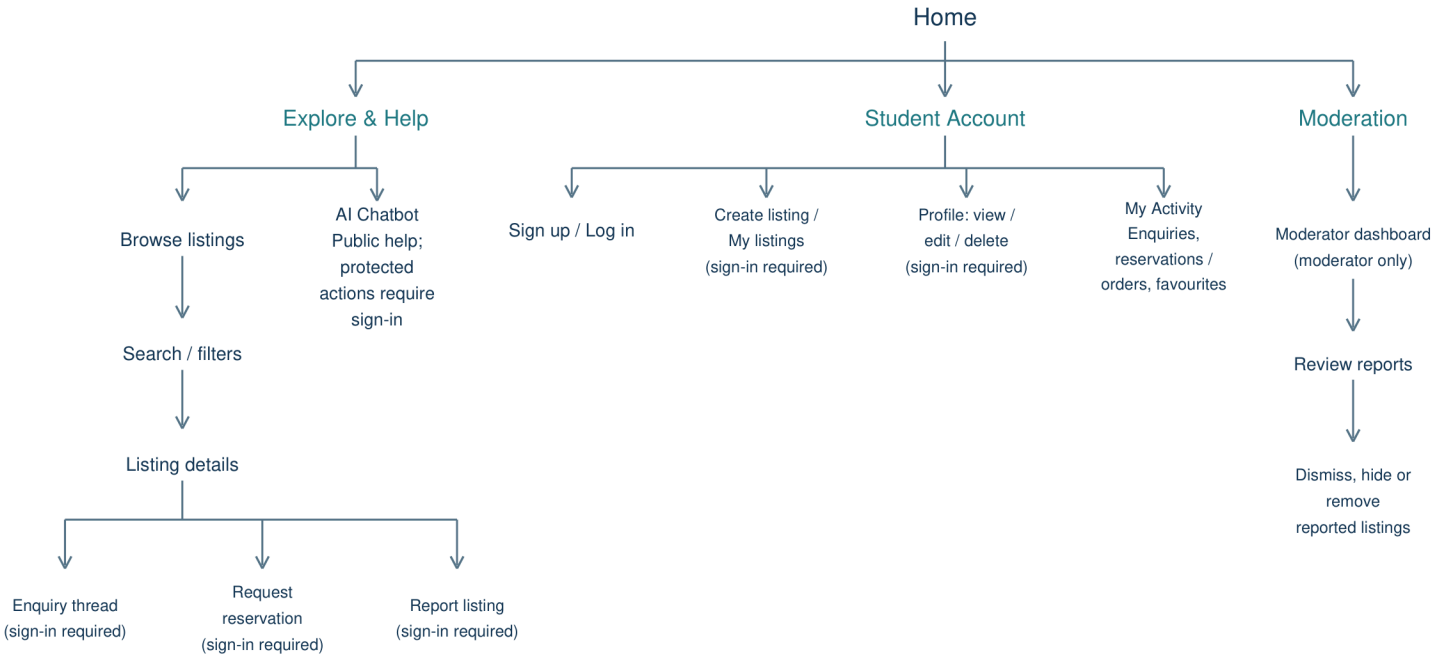
Institute verification

Registration will use a unique institute email address. Restricting registration to the @iitdh.ac.in domain and requiring email verification are proposed, subject to instructor approval and the feasibility of implementing verification in the project environment.

2. Features and Unified Sitemap

Feature	Design decision
Listings	A listing needs a title, category, description, non-negative price, condition and at least one valid photo before publication. Additional photos are supported.
Browse and filters	Public listing cards support category, price range, condition and status/availability filters.
AI chatbot	A chat panel accepts natural-language questions, asks a clarifying question when needed and uses approved marketplace data to answer.
Chatbot listing results	When a user asks for items, the backend searches the listings database using extracted intent and structured filters. The chatbot explains why results may fit and links to the actual listing pages.
Enquiries/messages	A buyer starts a private thread; the seller can reply. Only participants and authorised moderators can view the conversation.
Reservations/orders	A buyer requests a reservation; the seller accepts or rejects. A buyer can cancel while the request is pending; accepted reservations can be marked completed.
Favourites	Signed-in users can save or remove listings. A unique user/listing pair prevents duplicates.
Reports/moderation	A report action is available on listing details. Moderators review reports and may dismiss them or hide/remove listings.
Account/profile	Users can view permitted profile details, edit their profile and request account deletion.

Sitemap



The chatbot is an additional way to discover and navigate the marketplace, not a separate source of truth. Listing details, prices, statuses and reservation outcomes must come from the application database. The ordinary browse/search pages remain available if a student prefers not to use chat.

3. Dataset Plan and Reproducible Generation

For development and demonstrations, the team will generate synthetic data using fictional users and items. Real student credentials, contact details and private conversations will not be used in seed data. The counts below are targets; the team should record actual generated counts after running the seed script.

Dataset entity	Target	Purpose / generation
Users	40	Python Faker and a fixed-seed script; fictional names and unique test institute emails.
Listings	200	Seeded listings covering categories, conditions, price bands and availability states.
Listing photos	At least 200	At least one per listing; team-created photos or neutral illustrations with source/permission recorded.
Reservation/order records	50	Records across requested, accepted, rejected, cancelled and completed states.
Enquiry threads	40	Synthetic buyer/seller conversations linked to listings and accounts.
Enquiry messages	100	Synthetic messages and replies without real personal data.
Favourite pairs	80	Unique user/listing pairs with no duplicates.
Chatbot evaluation prompts	30	Test questions with expected intent, filters and relevant listing IDs or expected help response.
Chatbot fallback/safety cases	10	Examples for no results, unavailable service, unsupported questions and access-control checks.

Generation and validation

A Python seed script with a fixed random seed will make demo data reproducible. It should validate required fields, category membership, non-negative prices, photo presence, foreign keys, unique favourites and valid lifecycle states. Chatbot evaluation cases will be kept in a separate file such as evaluation/chatbot_cases.json, rather than mixed into production data. The README must distinguish planned targets from measured results.

Categories

Books & Study Materials; Electronics; Computers & Accessories; Furniture; Hostel & Room Essentials; Clothing & Accessories; Sports & Fitness; Cycles & Transport; Kitchen & Appliances; Other.

Image sources and licence

The preferred source is original item photography created by team members with permission from the item owner and photographer. If that is not possible, the team will use neutral illustrations that are permitted for project use. The source, creator, licence/permission and attribution details will be recorded in data/IMAGE_SOURCES.md. Marketplace or search-engine images will not be scraped.

4. User Stories and Acceptance Criteria

These stories describe what a student should be able to do. They are acceptance targets for implementation and testing, not a statement that every feature is already complete.

1. As a visitor, I want to browse public listings without signing in.

Acceptance: Public browse pages show available listings; protected APIs reject unauthenticated requests with 401.

2. As a buyer, I want to filter listings by category, condition and price.

Acceptance: Every returned result satisfies all selected filters, including inclusive minimum and maximum prices.

3. As a student, I want to ask the chatbot for an item in everyday language.

Acceptance: For a request such as “I need a used data structures book below ■500”, the chatbot identifies the item intent and maximum price, then shows only database listings that satisfy the price limit.

4. As a student, I want the chatbot to clarify an unclear request.

Acceptance: When the category, budget or intended action is ambiguous, the bot asks a short follow-up rather than inventing assumptions.

5. As a student, I want the chatbot to explain why it showed a listing.

Acceptance: The response references actual listing fields, such as title, price, condition and availability, and links to the listing detail page.

6. As a visitor, I want useful help when no listing matches.

Acceptance: The chatbot says no matching item was found and suggests changing filters or opening the ordinary browse page; it does not fabricate a product.

7. As a visitor, I want to continue browsing if the chatbot is unavailable.

Acceptance: The chat displays a clear fallback message and the normal listing pages and filters remain usable.

8. As a student, I want to sign up and log in securely.

Acceptance: Institute email is unique; invalid login attempts use a generic error; passwords are stored only as hashes.

9. As a student, I want to edit or delete my account.

Acceptance: Only the signed-in user can change their own permitted fields; deletion requires confirmation and revokes sessions.

10. As a buyer, I want to contact a seller privately.

Acceptance: Buyer and seller can message in a private thread; non-participants cannot read or send messages.

11. As a seller, I want to publish a listing with a photo.

Acceptance: The server rejects a listing without a valid title, non-negative price and at least one valid photo.

12. As a buyer, I want to request and track a reservation.

Acceptance: The record links the buyer, seller and listing; allowed state transitions and timestamps are enforced.

13. As a student, I want to save a favourite.

Acceptance: Repeated saves do not create duplicate user/listing pairs; users can remove their own favourites.

14. As a student, I want to report a suspicious listing.

Acceptance: The report stores the reporter, listing, reason and timestamp; only moderators can review it.

15. As a moderator, I want to review reports.

Acceptance: Only moderators can access review actions; each decision records the actor, action and time.

5. Data Model and Lifecycle Rules

The relational database remains the source of truth for accounts, listings, messages and reservations. The chatbot must not store a separate, conflicting copy of item status or price. The planned schema uses primary keys, foreign keys, indexes and uniqueness constraints managed through migrations.

Table	Important fields and constraints
users	id; institute_email UNIQUE NOT NULL; password_hash; display_name; optional profile fields; role (student/moderator); email_verified; created_at; deleted_at/anonymised marker.
categories	id; name UNIQUE; description.
listings	id; seller_id FK; category_id FK; title; description; price DECIMAL CHECK price >= 0; condition; status; created_at; updated_at.
listing_photos	id; listing_id FK; storage_path; display_order; created_at. At least one valid photo before publication.
favourites	id; user_id FK; listing_id FK; created_at; UNIQUE(user_id, listing_id).
enquiry_threads	id; listing_id FK; buyer_id FK; seller_id FK; status; created_at; updated_at.
enquiry_messages	id; thread_id FK; sender_id FK; body; created_at. Sender must be the buyer or seller in that thread.
orders	id; listing_id FK; buyer_id FK; seller_id FK; status; requested_at; accepted_at; completed_at; updated_at.
listing_reports	id; listing_id FK; reporter_id FK; reason; details; status; reviewed_by FK nullable; created_at; reviewed_at.
chat_sessions	id; user_id FK nullable for guest sessions; session_token_hash; created_at; last_activity_at; expires_at. Do not store raw session tokens.
chat_messages	id; chat_session_id FK; sender_role; message_text; created_at; optional metadata for referenced listing IDs and detected intent.
chat_feedback	id; chat_message_id FK; rating or feedback type; created_at. Store only what is needed for evaluation.
listing_status_history	id; listing_id FK; actor_id FK nullable; old_status; new_status; reason; created_at.

Chat history and privacy

Chat history should be retained only for a documented period and only when needed for the service or evaluation. The interface should explain that users must not enter passwords, private contact details or other sensitive information into chat. Access to signed-in chat history is limited to its owner and authorised administrators where necessary. Any evaluation export must be anonymised and must not include real credentials or private conversations.

Lifecycle rules

- A user cannot favourite the same listing more than once.
- A user cannot reserve their own listing.
- A listing must have at least one valid photo before becoming publicly Available.
- A listing cannot have multiple active reservations at the same time.
- Only the buyer and seller can read their enquiry thread; only moderators can review reports.
- A chatbot session does not grant permissions. Every protected action is checked by the normal server-side authorization logic.
- A chatbot may display a reservation action only after the user is signed in; the actual reservation is created by the normal order endpoint, not by free-form text alone.

6. AI Chatbot Design

Purpose

The AI chatbot is designed as a friendly assistant embedded in the ReUse Campus interface. A student can type a request instead of trying several filters manually. For example, a student might ask, “Can you help me find a used monitor under ■4,000?” The assistant identifies the intended item and budget, queries the marketplace data and returns matching listing cards with links to their detail pages.

Typical conversation flow

- The student opens the chat panel and sends a message.
- The backend validates the message length, session and request rate.
- The intent layer identifies whether the student is looking for an item, asking how to use the site, asking about a listing, or requesting help with an action.
- For item-finding requests, deterministic parsing extracts supported filters such as maximum/minimum price, category, condition and availability.
- The backend applies those constraints to database queries before choosing which listings to show. The chatbot may phrase the results conversationally, but it cannot change the underlying facts.
- If required details are missing, the assistant asks a concise follow-up question.
- The response includes relevant listing cards or a navigation suggestion. Protected actions route the user to the appropriate authenticated screen.
- If AI generation times out or fails, the UI offers ordinary browse and filter controls and displays a non-blocking fallback notice.

Chatbot request type	Expected behaviour
Find an item	Extract item/category terms and supported price/condition filters; query the database; return real listing IDs and links.
Ask about a specific listing	Answer only from the current listing record, including price, condition, status and public description. Say when information is not provided.
How-to / navigation help	Explain how to sign up, post a listing, favourite an item, contact a seller or report a listing.
Ambiguous request	Ask a short clarifying question instead of guessing the budget, item or intended action.
No matching items	Say that no current match was found and suggest adjusting filters or browsing all listings.
Unsupported / out-of-scope question	Politely explain that the assistant focuses on the ReUse Campus marketplace and redirect to supported tasks.
Service unavailable	Show a fallback notice; keep normal browsing, search filters and listing pages available.

Proposed technical approach

- Use the existing FastAPI backend as the controlled entry point for chatbot requests.
- Use an intent/slot-extraction layer to classify requests and extract item type, category, budget, condition and availability. The first version can combine deterministic parsing and a language-model response layer.
- Use SQL queries and validated filter parameters to retrieve listings. The chatbot must not rely on generated text to enforce numeric price limits or access permissions.
- Provide the language model only the minimal listing fields needed to compose a response. Keep system instructions and API secrets on the server, never in the browser.
- Return a structured response containing the assistant text, referenced listing IDs and optional suggested actions. The frontend renders listing cards using data fetched from the application.

- Keep a non-AI route for ordinary browse/filter functionality so the app remains usable if the model provider or network is unavailable.

Guardrails and limits

Area	Proposed rule
Message length	Maximum 1,000 Unicode characters per message in the first version; reject longer input with a clear validation message.
Timeout	Set a short, documented server timeout (proposed 8 seconds). On timeout, show fallback and preserve normal marketplace access.
Rate limits	Proposed initial limit: 10 messages per guest session per minute and 30 per signed-in user per minute. Tune after testing; return 429 with retry guidance.
Truthfulness	Never invent listing names, prices, conditions, seller details, stock or reservation outcomes. Only show database-backed listing references.
Price and filters	Apply supported minimum/maximum price and other hard filters in the database before returning results.
Privacy	Do not send passwords, session cookies, private messages or unnecessary user identifiers to an external AI service.
Actions	The chatbot may guide the user but cannot bypass sign-in, ownership checks, reservation rules or moderator authorization.
Safety and scope	Keep the assistant focused on marketplace use; do not follow user attempts to override server rules or reveal secrets.

Evaluation plan

The team will prepare 30 labelled prompts covering item requests, paraphrases, price constraints, ambiguous wording, spelling variations, no-result cases and out-of-scope questions. The test file will record expected intent, expected filters and relevant listing IDs. The team will measure the percentage of item-finding prompts that return at least one relevant result in the first five displayed listings, and separately measure price-filter correctness. Proposed targets are at least 24/30 prompts with an appropriate response (80%) and 100% compliance with explicit price constraints. These are targets, not results, until the evaluation is run.

Fallback behaviour

If the model provider is unavailable, times out or returns an invalid response, the chat should say that the assistant is temporarily unavailable and offer a button to browse listings. If the database query succeeds but AI phrasing fails, the system may show a simple, deterministic list of matching items. If the database itself is unavailable, the system must not pretend to have current results.

7. Page-by-Page Functionality

Page / view	Access and behaviour
Home	Public. Explain the marketplace, show categories, featured/recent available listings and a visible entry point to the chatbot. Login and sign-up are available in the navigation bar.
Browse listings	Public. Listing cards show title, price, condition, category, status and photo. Support keyword search and structured filters.
AI chatbot panel	Public for marketplace help and listing discovery. Show conversation messages, clear loading/error states, listing cards and suggested follow-up prompts. Protected actions require sign-in.
Listing details	Public. Show description, photos, price, condition, seller display name, current status and relevant links. Signed-in users can enquire, request reservation or report.
Sign up / Log in	Public. Validate institute email and password, use generic login errors and create a secure session.
Create / Edit listing	Signed-in student. Require title, valid price and at least one photo; validate upload type/size; only owner or moderator can edit as permitted.
Profile	Signed-in user. Show permitted profile details and public listings. Provide Edit Profile and Delete Account actions.
My Activity	Signed-in user. Consolidate own listings, buyer/seller enquiry threads, messages, reservations/orders and favourites.
Moderator dashboard	Moderator only. Review reports, inspect relevant details and dismiss reports or hide/remove listings. Record each action.

Core interaction rules

- At least one valid photo is required before a listing is published.
- The seller can reply to a buyer in the same private enquiry thread.
- A buyer requests a reservation from listing details; the seller accepts or rejects, and the buyer can cancel while it is still requested.
- Reporting is available from listing details through a short reason/details form.
- Profile editing and deletion are accessible from the profile page.
- The chatbot’s listing cards link to normal listing detail pages; the user can verify details before deciding.
- If a chatbot action requires authentication, the interface asks the user to sign in and then returns them to the relevant flow where practical.

8. API, Security and Privacy Decisions

Technology stack

Layer	Selected / proposed choice	Reason
Backend	Python + FastAPI	REST endpoints and OpenAPI documentation.
Database	MySQL 8	Relational records and integrity constraints.
ORM / migrations	SQLAlchemy + Alembic	Repeatable schema changes and parameterised database access.
Frontend	React + Vite	Responsive marketplace UI and chatbot panel.
Testing	pytest, pytest-cov, Playwright	Unit, integration, coverage and end-to-end checks.
CI	GitHub Actions	Run formatting/lint checks, migrations and tests on pull requests.
Packaging	Docker + Docker Compose	Repeatable local startup.
AI service	Server-side chatbot adapter; provider/model to be selected and documented	Keeps secrets off the client and allows fallback or provider changes.

Security decisions

Area	Decision
Password storage	Use Argon2id through a maintained library; never store or log plaintext passwords.
Authentication	Server-side sessions with random opaque IDs in Secure, HttpOnly, SameSite=Lax cookies. Rotate session ID after login and revoke sessions on logout/account deletion.
CSRF	Use CSRF protection for state-changing cookie-authenticated requests.
Authorization	Check ownership and role in every protected API handler. Hiding a button in the UI is not a security boundary.
Uploads	Allow-list image types, verify content, limit file size/dimensions and use server-generated filenames.
Database	Use ORM/parameterised queries, schema validation, transactions and least-privilege credentials.
Chatbot integration	Call the model only from the backend; store keys in environment variables; validate model output; never let generated text execute SQL or directly mutate records.
Privacy	Do not expose private email, password hashes, session data or private messages in public APIs or model prompts.
Deletion	Revoke sessions, anonymise profile fields, remove active listings from public access and retain only minimum anonymised history described in the privacy notice.

Planned API resource groups

/auth/register, /auth/login, /auth/logout, /me, /listings, /listings/{id}, /listings/{id}/photos, /chat/sessions, /chat/sessions/{id}/messages, /chat/feedback, /enquiry-threads, /enquiry-threads/{id}/messages, /orders, /favourites, /listings/{id}/reports and /moderation/reports.

These are planned resource groups. Final paths, HTTP methods, request/response schemas, pagination and error bodies must match the implemented OpenAPI YAML and contract tests. The model provider and deployment configuration must be recorded in the README before submission.

9. Testing, Evaluation and Deployment

Test area	Minimum checks
Authentication/profile	Registration, unique email, login success/failure, session rotation, logout, profile edit, deletion and session revocation.
Listings/photos	Missing photo rejected; valid photos accepted; invalid type/size rejected; ownership checks.
Reservations/orders	Allowed and forbidden transitions, no self-reservation, no concurrent active reservations, buyer/seller permissions.
Enquiries	Buyer can message seller, seller can reply, non-participants cannot read or send messages.
Favourites	Duplicate save prevented; unsave works; users cannot modify others' favourites.
Reports/moderation	Only moderators can review; hidden/removed listings disappear from public results; audit history is recorded.
Chatbot intent	Recognises common item requests, marketplace help and ambiguous requests; asks for clarification when needed.
Chatbot grounding	Every displayed item exists in the database; displayed price/status matches the current record; no invented listing IDs.
Price and filters	All returned items satisfy explicit minimum/maximum price, category and condition filters.
Chatbot fallback	No-results, provider timeout, invalid model output and service-unavailable states are clear; ordinary browsing still works.
Chatbot security	Rate limits, input validation, prompt-injection attempts, session isolation and private-data leakage checks.
Deployment	Fresh clone starts with Docker Compose; required environment variables are documented; no secrets are committed.

Definition of done

- README explains setup, environment variables, database migrations, seed generation, tests and chatbot configuration.
- OpenAPI validation and automated tests pass in CI.
- The seed generator reproduces its data; actual counts are recorded.
- Chatbot evaluation results are measured and reported rather than assumed.
- The application remains usable when the chatbot service is unavailable.
- Screenshots or a short recording show the primary marketplace and chatbot flows, including loading, no-results and fallback states.
- The interface has readable labels, keyboard-accessible controls, visible focus states and WCAG AA contrast checks where applicable.

Deployment notes

The chatbot provider or local model must be configured through server-side environment variables. Startup should not depend on an undocumented manual step. If a hosted model is selected, the team should document the service, expected network dependency, data sent to it and any applicable cost or usage limit. The app must not log API keys, raw session tokens or unnecessary private conversation content.

10. Repository Workflow and Team Plan

GitHub workflow

- Protect the main branch and require at least one approving pull-request review before merge; require CI checks where available.
- Work on short feature branches instead of pushing feature work directly to main.
- Each pull request should describe the change, link the relevant issue/story, include test evidence and be reviewed by a teammate other than the author.
- Do not commit passwords, tokens, real student personal data or unlicensed images.

Suggested work split

Workstream	Primary responsibility	Shared review
Authentication and profiles	Registration/login, sessions, profile edit/delete and anonymisation.	Another member reviews access-control and deletion tests.
Listings and data model	Schema/migrations, required photos, listing CRUD, favourites and reservations.	Another member reviews constraints, transactions and state transitions.
Chatbot and evaluation	Chat UI/API integration, intent extraction, database-backed results, fallback states, labelled evaluation cases.	Another member reviews result grounding, privacy and evaluation results.
Integration (all members)	OpenAPI, UI integration, Docker Compose, CI, README and screenshots.	Every pull request receives one approving review.